WebSocket Streams for Binance

General WSS information

- The base endpoint is: **wss://stream.binance.com:9443** or **wss://stream.binance.com:443**.
- Streams can be accessed either in a single raw stream or in a combined stream.
- Raw streams are accessed at **/ws/<streamName>**
- Combined streams are accessed at **/stream?streams=<streamName1>/<streamName2>/<streamName3>**
- Combined stream events are wrapped as follows: **{"stream":"<streamName>","data":<rawPayload>}**
- All symbols for streams are **lowercase**
- A single connection to **stream.binance.com** is only valid for 24 hours; expect to be disconnected at the 24 hour mark
- The WebSocket server will send a `ping frame` every 20 seconds.
  - If the WebSocket server does not receive a `pong frame` back from the connection within a minute the connection will be disconnected.
  - When you receive a ping, you must send a pong with a copy of ping's payload as soon as possible.
  - Unsolicited `pong frames` are allowed, but will not prevent disconnection. **It is recommended that the payload for these pong frames are empty.**
- The base endpoint **wss://data-stream.binance.vision** can be subscribed to receive **only** market data messages.
  User data stream is **NOT** available from this URL.
- All time and timestamp related fields are **milliseconds by default**. To receive the information in microseconds, please add the parameter `timeUnit=MICROSECOND or timeUnit=microsecond` in the URL.
  - For example: `/stream?streams=btcusdt@trade&timeUnit=MICROSECOND`
- If your request contains a symbol name containing non-ASCII characters, then the stream events may contain non-ASCII characters encoded in UTF-8.
- [All Market Mini Tickers Stream](#all-market-mini-tickers-stream and All Market Rolling Window Statistics Streams events may contain non-ASCII characters encoded in UTF-8.

WebSocket Limits

- WebSocket connections have a limit of 5 incoming messages per second. A message is considered:
  - A PING frame
  - A PONG frame
  - A JSON controlled message (e.g. subscribe, unsubscribe)
- A connection that goes beyond the limit will be disconnected; IPs that are repeatedly disconnected may be banned.
- A single connection can listen to a maximum of 1024 streams.
- There is a limit of **300 connections per attempt every 5 minutes per IP**.

Live Subscribing/Unsubscribing to streams

- The following data can be sent through the WebSocket instance in order to subscribe/unsubscribe from streams. Examples can be seen below.
- The `id` is used as an identifier to uniquely identify the messages going back and forth. The following formats are accepted:
  - 64-bit signed integer
  - alphanumeric strings; max length 36
  - `null`
- In the response, if the `result` received is `null` this means the request sent was a success for non-query requests (e.g. Subscribing/Unsubscribing).

Subscribe to a stream

- Request

```
{
    "method": "SUBSCRIBE",
    "params": ["btcusdt@aggTrade", "btcusdt@depth"],
    "id": 1
}
```

- Response

```
{
    "result": null,
```

```
    "id": 1
}
```

Unsubscribe to a stream

- Request

```
{
    "method": "UNSUBSCRIBE",
    "params": ["btcusdt@depth"],
    "id": 312
}
```

- Response

```
{
    "result": null,
    "id": 312
}
```

Listing Subscriptions

- Request

```
{
    "method": "LIST_SUBSCRIPTIONS",
    "id": 3
}
```

- Response

```
{
    "result": ["btcusdt@aggTrade"],
    "id": 3
}
```

Setting Properties

Currently, the only property that can be set is whether `combined` stream payloads are enabled or not. The combined property is set to `false` when connecting using `/ws/` ("raw streams") and `true` when connecting using `/stream/`.

- Request

```
{
    "method": "SET_PROPERTY",
    "params": ["combined", true],
    "id": 5
}
```

- Response

```
{
    "result": null,
    "id": 5
}
```

Retrieving Properties

- Request

```
{
    "method": "GET_PROPERTY",
    "params": ["combined"],
    "id": 2
}
```

- Response

```
{
    "result": true, // Indicates that combined is set to true.
    "id": 2
}
```

Error Messages

Error Message	Description
{"code": 0, "msg": "Unknown property","id": %s}	Parameter used in the `SET_PROPERTY` or `GET_PROPERTY` was invalid
{"code": 1, "msg": "Invalid value type: expected Boolean"}	Value should only be `true` or `false`
{"code": 2, "msg": "Invalid request: property name must be a string"}	Property name provided was invalid
{"code": 2, "msg": "Invalid request: request ID must be an unsigned integer"}	Parameter `id` had to be provided or the value provided in the `id` parameter is an unsupported type
{"code": 2, "msg": "Invalid request: unknown variant %s, expected one of `SUBSCRIBE`, `UNSUBSCRIBE`, `LIST_SUBSCRIPTIONS`, `SET_PROPERTY`, `GET_PROPERTY` at line 1 column 28"}	Possible typo in the provided method or provided method was neither of the expected values
{"code": 2, "msg": "Invalid request: too many parameters"}	Unnecessary parameters provided in the data
{"code": 2, "msg": "Invalid request: property name must be a string"}	Property name was not provided
{"code": 2, "msg": "Invalid request: missing field `method` at line 1 column 73"}	`method` was not provided in the data
{"code":3,"msg":"Invalid JSON: expected value at line %s column %s"}	JSON data sent has incorrect syntax.

Detailed Stream information

Reference Price Streams

Stream Name: <symbol>@referencePrice

Update Speed: 1000ms

Payload:

```
{
  "e": "referencePrice",  // eventType
```

```
  "s": "BAZUSD",          // symbol
  "r": "1.00",            // reference price (null if no reference price)
  "t": 1770313263917      // engine timestamp when reference price was valid
}
```

Aggregate Trade Streams

The Aggregate Trade Streams push trade information that is aggregated for a single taker order.

Stream Name: <symbol>@aggTrade

Update Speed: Real-time

Payload:

```
{
    "e": "aggTrade",        // Event type
    "E": 1672515782136,     // Event time
    "s": "BNBBTC",          // Symbol
    "a": 12345,             // Aggregate trade ID
    "p": "0.001",           // Price
    "q": "100",             // Quantity
    "f": 100,               // First trade ID
    "l": 105,               // Last trade ID
    "T": 1672515782136,     // Trade time
    "m": true,              // Is the buyer the market maker?
    "M": true               // Ignore
}
```

Trade Streams

The Trade Streams push raw trade information; each trade has a unique buyer and seller.

Stream Name: <symbol>@trade

Update Speed: Real-time

Payload:

```
{
    "e": "trade",           // Event type
    "E": 1672515782136,     // Event time
    "s": "BNBBTC",          // Symbol
    "t": 12345,             // Trade ID
    "p": "0.001",           // Price
    "q": "100",             // Quantity
```

```
    "T": 1672515782136,     // Trade time
    "m": true,              // Is the buyer the market maker?
    "M": true               // Ignore
}
```

Kline/Candlestick Streams for UTC

The Kline/Candlestick Stream push updates to the current klines/candlestick every second in `UTC+0` timezone

Kline/Candlestick chart intervals:

s-> seconds; m -> minutes; h -> hours; d -> days; w -> weeks; M -> months

- 1s
- 1m
- 3m
- 5m
- 15m
- 30m
- 1h
- 2h
- 4h
- 6h
- 8h
- 12h
- 1d
- 3d
- 1w
- 1M

Stream Name: <symbol>@kline_<interval>

Update Speed: 1000ms for `1s`, 2000ms for the other intervals

Payload:

```
{
    "e": "kline",              // Event type
    "E": 1672515782136,        // Event time
```

```
    "s": "BNBBTC",              // Symbol
    "k": {
        "t": 1672515780000,     // Kline start time
        "T": 1672515839999,     // Kline close time
        "s": "BNBBTC",          // Symbol
        "i": "1m",              // Interval
        "f": 100,               // First trade ID
        "L": 200,               // Last trade ID
        "o": "0.0010",          // Open price
        "c": "0.0020",          // Close price
        "h": "0.0025",          // High price
        "l": "0.0015",          // Low price
        "v": "1000",            // Base asset volume
        "n": 100,               // Number of trades
        "x": false,             // Is this kline closed?
        "q": "1.0000",          // Quote asset volume
        "V": "500",             // Taker buy base asset volume
        "Q": "0.500",           // Taker buy quote asset volume
        "B": "123456"           // Ignore
    }
}
```

Kline/Candlestick Streams with timezone offset

The Kline/Candlestick Stream push updates to the current klines/candlestick every second in `UTC+8` timezone

Kline/Candlestick chart intervals:

Supported intervals: See `Kline/Candlestick chart intervals`

UTC+8 timezone offset:

- Kline intervals open and close in the `UTC+8` timezone. For example the `1d` klines will open at the beginning of the `UTC+8` day, and close at the end of the `UTC+8` day.
- Note that `E` (event time), `t` (start time) and `T` (close time) in the payload are Unix timestamps, which are always interpreted in UTC.

Stream Name: <symbol>@kline_<interval>@+08:00

Update Speed: 1000ms for `1s`, 2000ms for the other intervals

Payload:

```
{
    "e": "kline",               // Event type
    "E": 1672515782136,         // Event time
    "s": "BNBBTC",              // Symbol
```

```
    "k": {
        "t": 1672515780000,     // Kline start time
        "T": 1672515839999,     // Kline close time
        "s": "BNBBTC",          // Symbol
        "i": "1m",              // Interval
        "f": 100,               // First trade ID
        "L": 200,               // Last trade ID
        "o": "0.0010",          // Open price
        "c": "0.0020",          // Close price
        "h": "0.0025",          // High price
        "l": "0.0015",          // Low price
        "v": "1000",            // Base asset volume
        "n": 100,               // Number of trades
        "x": false,             // Is this kline closed?
        "q": "1.0000",          // Quote asset volume
        "V": "500",             // Taker buy base asset volume
        "Q": "0.500",           // Taker buy quote asset volume
        "B": "123456"           // Ignore
    }
}
```

Individual Symbol Mini Ticker Stream

24hr rolling window mini-ticker statistics. These are NOT the statistics of the UTC day, but a 24hr rolling window for the previous 24hrs.

Stream Name: <symbol>@miniTicker

Update Speed: 1000ms

Payload:

```
{
    "e": "24hrMiniTicker",     // Event type
    "E": 1672515782136,        // Event time
    "s": "BNBBTC",             // Symbol
    "c": "0.0025",             // Close price
    "o": "0.0010",             // Open price
    "h": "0.0025",             // High price
    "l": "0.0010",             // Low price
    "v": "10000",              // Total traded base asset volume
    "q": "18"                  // Total traded quote asset volume
}
```

All Market Mini Tickers Stream

24hr rolling window mini-ticker statistics for all symbols that changed in an array. These are NOT the statistics of the UTC day, but a 24hr rolling window for the previous 24hrs. Note that only tickers that have changed will be present in the array.

Stream Name: !miniTicker@arr

Update Speed: 1000ms

Payload:

```
[
    {
        // Same as <symbol>@miniTicker payload
    }
]
```

Individual Symbol Ticker Streams

24hr rolling window ticker statistics for a single symbol. These are NOT the statistics of the UTC day, but a 24hr rolling window for the previous 24hrs.

Stream Name: <symbol>@ticker

Update Speed: 1000ms

Payload:

```
{
    "e": "24hrTicker",      // Event type
    "E": 1672515782136,     // Event time
    "s": "BNBBTC",          // Symbol
    "p": "0.0015",          // Price change
    "P": "250.00",          // Price change percent
    "w": "0.0018",          // Weighted average price
    "x": "0.0009",          // First trade(F)-1 price (first trade before the 24hr rolling
window)
    "c": "0.0025",          // Last price
    "Q": "10",              // Last quantity
    "b": "0.0024",          // Best bid price
    "B": "10",              // Best bid quantity
    "a": "0.0026",          // Best ask price
    "A": "100",             // Best ask quantity
    "o": "0.0010",          // Open price
    "h": "0.0025",          // High price
    "l": "0.0010",          // Low price
    "v": "10000",           // Total traded base asset volume
    "q": "18",              // Total traded quote asset volume
    "O": 0,                 // Statistics open time
```

```
    "C": 86400000,          // Statistics close time
    "F": 0,                 // First trade ID
    "L": 18150,             // Last trade Id
    "n": 18151              // Total number of trades
}
```

Individual Symbol Rolling Window Statistics Streams

Rolling window ticker statistics for a single symbol, computed over multiple windows.

Stream Name: <symbol>@ticker_<window_size>

Window Sizes: 1h,4h,1d

Update Speed: 1000ms

Note: This stream is different from the <symbol>@ticker stream. The open time `"O"` always starts on a minute, while the closing time `"C"` is the current time of the update. As such, the effective window might be up to 59999ms wider than <window_size>.

Payload:

```
{
    "e": "1hTicker",        // Event type
    "E": 1672515782136,     // Event time
    "s": "BNBBTC",          // Symbol
    "p": "0.0015",          // Price change
    "P": "250.00",          // Price change percent
    "o": "0.0010",          // Open price
    "h": "0.0025",          // High price
    "l": "0.0010",          // Low price
    "c": "0.0025",          // Last price
    "w": "0.0018",          // Weighted average price
    "v": "10000",           // Total traded base asset volume
    "q": "18",              // Total traded quote asset volume
    "O": 0,                 // Statistics open time
    "C": 1675216573749,     // Statistics close time
    "F": 0,                 // First trade ID
    "L": 18150,             // Last trade Id
    "n": 18151              // Total number of trades
}
```

All Market Rolling Window Statistics Streams

Rolling window ticker statistics for all market symbols, computed over multiple windows. Note that only tickers that have changed will be present in the array.

Stream Name: !ticker_<window-size>@arr

Window Size: 1h,4h,1d

Update Speed: 1000ms

Payload:

```
[
    {
        // Same as <symbol>@ticker_<window_size> payload,
        // one for each symbol updated within the interval.
    }
]
```

Individual Symbol Book Ticker Streams

Pushes any update to the best bid or ask's price or quantity in real-time for a specified symbol. Multiple `<symbol>@bookTicker` streams can be subscribed to over one connection.

Stream Name: <symbol>@bookTicker

Update Speed: Real-time

Payload:

```
{
    "u": 400900217,         // order book updateId
    "s": "BNBUSDT",         // symbol
    "b": "25.35190000",     // best bid price
    "B": "31.21000000",     // best bid qty
    "a": "25.36520000",     // best ask price
    "A": "40.66000000"      // best ask qty
}
```

Average Price

Average price streams push changes in the average price over a fixed time interval.

Stream Name: <symbol>@avgPrice

Update Speed: 1000ms

Payload:

```
{
    "e": "avgPrice",           // Event type
    "E": 1693907033000,        // Event time
    "s": "BTCUSDT",            // Symbol
    "i": "5m",                 // Average price interval
    "w": "25776.86000000",     // Average price
    "T": 1693907032213         // Last trade time
}
```

Partial Book Depth Streams

Top **<levels>** bids and asks, pushed every second. Valid **<levels>** are 5, 10, or 20.

Stream Names: <symbol>@depth<levels> OR <symbol>@depth<levels>@100ms

Update Speed: 1000ms or 100ms

Payload:

```
{
    "lastUpdateId": 160,     // Last update ID
    "bids": [                // Bids to be updated
        [
            "0.0024",        // Price level to be updated
            "10"             // Quantity
        ]
    ],
    "asks": [                // Asks to be updated
        [
            "0.0026",        // Price level to be updated
            "100"            // Quantity
        ]
    ]
}
```

Diff. Depth Stream

Order book price and quantity depth updates used to locally manage an order book.

Stream Name: <symbol>@depth OR <symbol>@depth@100ms

Update Speed: 1000ms or 100ms

Payload:

```
{
    "e": "depthUpdate",     // Event type
    "E": 1672515782136,     // Event time
    "s": "BNBBTC",          // Symbol
    "U": 157,               // First update ID in event
    "u": 160,               // Final update ID in event
    "b": [                  // Bids to be updated
        [
            "0.0024",       // Price level to be updated
            "10"            // Quantity
        ]
    ],
    "a": [                  // Asks to be updated
        [
            "0.0026",       // Price level to be updated
            "100"           // Quantity
        ]
    ]
}
```

How to manage a local order book correctly

1. Open a WebSocket connection to `wss://stream.binance.com:9443/ws/bnbbtc@depth`.
2. Buffer the events received from the stream. Note the `U` of the first event you received.
3. Get a depth snapshot from `https://api.binance.com/api/v3/depth?symbol=BNBBTC&limit=5000`.
4. If the `lastUpdateId` from the snapshot is strictly less than the `U` from step 2, go back to step 3.
5. In the buffered events, discard any event where `u` is <= `lastUpdateId` of the snapshot. The first buffered event should now have `lastUpdateId` within its `[U;u]` range.
6. Set your local order book to the snapshot. Its update ID is `lastUpdateId`.
7. Apply the update procedure below to all buffered events, and then to all subsequent events received.

To apply an event to your local order book, follow this update procedure:

1. Decide whether the update event can be applied:
   - If the event last update ID (`u`) is less than the update ID of your local order book, ignore the event.
   - If the event first update ID (`U`) is greater than the update ID of your local order book + 1, you have missed some events.
     Discard your local order book and restart the process from the beginning.
   - Normally, `U` of the next event is equal to `u + 1` of the previous event.
2. For each price level in bids (`b`) and asks (`a`), set the new quantity in the order book:

- If the price level does not exist in the order book, insert it with new quantity.
- If the quantity is zero, remove the price level from the order book.

3. Set the order book update ID to the last update ID (`u`) in the processed event.

> [!NOTE] Since depth snapshots retrieved from the API have a limit on the number of price levels (5000 on each side maximum), you won't learn the quantities for the levels outside of the initial snapshot unless they change.
> So be careful when using the information for those levels, since they might not reflect the full view of the order book.
> However, for most use cases, seeing 5000 levels on each side is enough to understand the market and trade effectively.